

End-to-End Differential Privacy in Training Deep Neural Network Classifiers

Huaiyuan Rao

*School of Electrical and Computer Engineering,
Georgia Institute of Technology
Atlanta, GA 30332, USA*

HRAO43@GATECH.EDU

Calvin Hawkins

*School of Electrical and Computer Engineering,
Georgia Institute of Technology
Atlanta, GA 30332, USA*

CHAWKINS64@GATECH.EDU

Alexander Benvenuti

*School of Electrical and Computer Engineering,
Georgia Institute of Technology
Atlanta, GA 30332, USA*

ABENVENUTI3@GATECH.EDU

Matthew Hale

*School of Electrical and Computer Engineering,
Georgia Institute of Technology
Atlanta, GA 30332, USA*

MATTHALE@GATECH.EDU

Abstract

Differentially private machine learning enables model training on sensitive data while ensuring that individual data is unlikely to be recoverable from the parameters of the resulting model. However, existing work often privatizes both training inputs and their labels, and these protections may be conservative when labels are public or can be safely made public. Therefore, in this work we propose a novel private training framework that instead privatizes training inputs while keeping labels public. We consider neural networks with softmax output layers, and thus the mapping from training inputs to the output of the softmax layer is a mapping onto the unit simplex. We randomize softmax outputs during training by applying the Dirichlet mechanism to enforce differential privacy for the training inputs, hence the “end-to-end” label. Because training data is reused across multiple training epochs, we use the notion of Rényi differential privacy to formulate tight bounds on the strength of privacy provided by the Dirichlet mechanism across repeated uses. We show empirically that we attain new state-of-the-art accuracy when training from scratch on CIFAR10, MNIST, MedMNIST, FashionMNIST, and SVHN across all privacy budgets evaluated. Notably, when implementing (ϵ, δ) -differential privacy with $\delta = 10^{-5}$, we improve the prior state-of-the-art accuracy from 78.37% to 88.17% at $\epsilon = 4$ on CIFAR10, and our approach has 82.96% accuracy even for $\epsilon = 1$, which significantly outperforms prior work.

Keywords: differential privacy, Rényi differential privacy, Dirichlet mechanism, unit simplex, classification

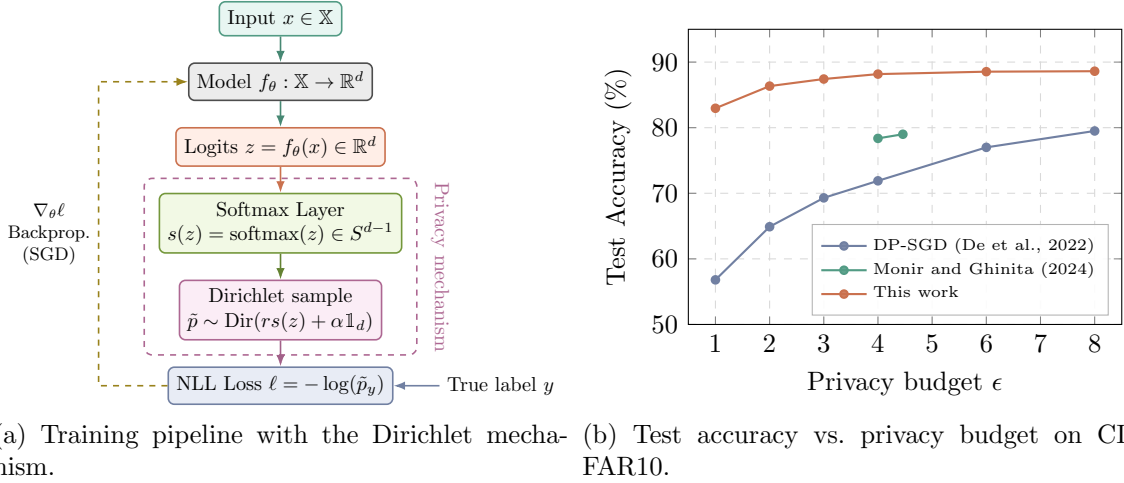


Figure 1: Overview of the proposed private training framework. (a) The model f_θ first outputs logits that pass through a softmax layer, then a private element of the unit simplex is sampled from a Dirichlet distribution, and it is used to compute the Negative Log-likelihood (NLL) loss. Then the loss is used to compute gradients and backpropagate to update the model parameters θ . (b) On CIFAR10, our method consistently outperforms DP-SGD (De et al., 2022) and Monir and Ghinita (2024) across all privacy budgets ϵ . For example, our method achieves over 82% accuracy when $\epsilon = 1$, while the prior work (Monir and Ghinita, 2024) achieves 79% accuracy when $\epsilon = 4.46$.

1 Introduction

Machine learning (ML) models trained on sensitive information such as medical images, biometric data, or financial records are vulnerable to privacy attacks. Examples include reconstruction attacks (Rigaki and Garcia, 2023), which recover sensitive data from learned parameters, or membership inference attacks (Shokri et al., 2017), which determine whether a specific individual’s record was included in a training data set. Other types of attacks are described in (Fredrikson et al., 2015; Ganju et al., 2018; Melis et al., 2019). Differential privacy (Dwork et al., 2014) is a general-purpose privacy framework that protects against many such attacks, and differentially private ML has been used to train models in such a way that an adversary cannot reliably use a model to draw accurate conclusions about the data used in training it. In this paper, we focus on using differential privacy in training deep neural networks for classification tasks.

In some real-world applications, training inputs and the labels for those inputs do not have the same privacy risks. For example, in medical imaging, chest X-rays used for pneumonia detection may contain highly sensitive patient biometric information (Packhäuser et al., 2022), whereas the associated diagnostic labels may already be accessible via public health registries (Wang et al., 2017). In recommender systems, users publicly disclose their preferences through item ratings, while their personal social and demographic information (e.g., gender, political affiliation, and ethnicity) may be considered sensitive (Bhagat et al.,

2014). In financial applications, individual transaction histories are sensitive, while the categories such as risk tiers are often publicly defined. For example, in credit underwriting, borrowers are often publicly grouped into established tiers like “Prime”, “Near-Prime”, or “Subprime” (Bazarbash, 2019). In these and other applications, the training inputs often contain sensitive information about an individual, while the labels of that data are not sensitive and/or may already be public. Motivated by such applications, the question we address is the following:

Can we train ML models that provide differential privacy to sensitive training inputs while maintaining high accuracy of the models?

In this paper, we introduce a novel differentially private learning framework that injects calibrated randomness into the outputs of softmax layers, which enables model training with private neural network inputs and public labels. Our approach decomposes the training process into two stages: (i) computing a distribution that enforces a desired level of differential privacy, and (ii) randomizing softmax outputs at every training step by using the Dirichlet mechanism (Ponnoprat, 2023), which perturbs softmax outputs when computing the training loss in order to enforce differential privacy. An illustration of the proposed framework is shown in Figure 1a. As shown in Figure 1b, our method significantly outperforms DP-SGD (De et al., 2022) and other prior work (Monir and Ghinita, 2024) for all of the privacy budgets that were evaluated.

1.1 Related Work

Differentially private stochastic gradient descent (DP-SGD) (Abadi et al., 2016) is a widely used differentially private ML algorithm, which clips per-sample gradients and adds calibrated Gaussian noise to the gradient at each training step. While DP-SGD provides strong privacy guarantees to both training inputs and their labels, it introduces a substantial privacy-utility tradeoff: even at moderate privacy budgets, there is a large accuracy gap between models trained with DP-SGD and their non-private baselines. Many follow-up works have sought to reduce the privacy-utility gap of DP-SGD (De et al., 2022; Tramer and Boneh, 2020; Bu et al., 2023; Sander et al., 2023; Papernot et al., 2021; Cheng et al., 2022). One recent body of work has privatized only labels while leaving training inputs public, which has been termed “Label-DP” (Ghazi et al., 2021; Malek Esmaeili et al., 2021; Busa-Fekete et al., 2023; Ghazi et al., 2023; Esfandiari et al., 2022). By assuming that training inputs are non-sensitive, Label-DP reduces the amount of noise injected during training and achieves substantially better accuracy than DP-SGD.

The complementary case, in which inputs are sensitive but labels are public, has received less attention. Several works (Fan, 2018; Croft et al., 2021; Luo et al., 2025; Xue et al., 2021) privatize input data before applying a training procedure, rather than developing a new training procedure that is designed to enforce privacy. For example, Fan (2018) introduces a differentially private pixelization method that partitions an image into grid cells and perturbs each cell’s average pixel value with Laplace noise. However, the methods in (Fan, 2018; Croft et al., 2021; Luo et al., 2025; Xue et al., 2021) release a private image rather than a trained ML model, and their evaluations focus on re-identification resistance rather than classification accuracy. The goals of that work are different from the current paper, and a direct comparison would be ill-defined. The closest prior work to ours is Monir

and Ghinita (2024), which addresses the setting of sensitive inputs and public labels by injecting calibrated Gaussian noise into the penultimate layer of a deep neural network in a way similar to DP-SGD. However, the variance of privacy noise scales with layer width, which can harm accuracy, and per-sample gradient clipping introduces additional computational cost. We show in Section 4.2 that our approach provides a 45.31% reduction in the classification error on CIFAR10 relative to Monir and Ghinita (2024) when $\epsilon = 4.0$.

Privacy amplification by subsampling (Wang et al., 2019; Mironov et al., 2019) allows DP-SGD-style methods to use the moments accountant to obtain privacy guarantees over many training steps that are stronger than standard composition-type results for differential privacy. The core idea is to apply a privacy mechanism to randomly sampled minibatches from data sets and track the log moment generating function of the privacy loss. Composing these moments ensures a tight bound on the accumulated privacy loss across all training steps. Most existing work based on subsampling uses the Gaussian mechanism, which is the standard mechanism used in DP-SGD. To the best of our knowledge, there is no prior work that has used the Dirichlet mechanism (Ponnoprat, 2023) under subsampling. Our work does so, and thus we address not only the setting of learning from sensitive inputs with public labels, but we also provide the first subsampled privacy analysis of the Dirichlet mechanism.

1.2 Summary of Contributions

In this work, we make the following contributions:

- We propose a novel differentially private training framework that generates private softmax outputs using the Dirichlet mechanism (Algorithm 1).
- We provide a formal privacy analysis using subsampling amplification that gives a tight bound on the privacy guarantees of Algorithm 1 (Theorem 10).
- We show that, on average, the implementation of privacy preserves the direction of gradients while scaling their magnitude, and we use this result to demonstrate how to select learning rates to negate this scaling (Theorem 11).
- We empirically evaluate our method on several benchmark data sets and show that it outperforms prior work across all evaluated privacy budgets (Section 4). A depiction of these results is shown in Figure 1b. Our approach provides a 45.31% reduction in the classification error on CIFAR10 relative to the method in (Monir and Ghinita, 2024) when $\epsilon = 4.0$.

1.3 Notation

We use $\mathbb{R}$ to denote the real numbers, $\mathbb{R}_{\geq 0}$ to denote the non-negative real numbers, and $\mathbb{N}$ to denote the positive integers. For $n \in \mathbb{N}$, we use $[n]$ to denote the set $\{1, \dots, n\}$. We use $|S|$ to denote the cardinality of a finite set S . For $z \in \mathbb{R}^d$, we use $s(z)$ to denote the softmax function with input z , where the i^{th} component of the output of the softmax function is defined as $s_i(z) = \frac{\exp(z_i)}{\sum_{j \in [d]} \exp(z_j)}$. For $x > 0$, the gamma function is defined as $\Gamma(x) = \int_0^\infty t^{x-1} e^{-t} dt$, and we write $\psi(x) = \frac{d}{dx} \log(\Gamma(x))$ and $\psi'(x) = \frac{d^2}{dx^2} \log(\Gamma(x))$ for the digamma and trigamma functions, respectively. We use D_{KL} to denote the Kullback–Leibler (KL) divergence. For

$v \in \mathbb{R}_{\geq 0}^d$ the multivariate beta function is defined as $B(v) = \prod_{i=1}^d \Gamma(v_i) / \Gamma\left(\sum_{i=1}^d v_i\right)$. For $q \in (0, 1)$, we use $\text{Bernoulli}(q)$ to denote the Bernoulli distribution with parameter q , where a random variable drawn from this distribution takes the value 1 with probability q and takes the value 0 with probability $1 - q$. For a non-empty, finite set S , we use $\text{Uniform}(S)$ to denote the uniform distribution over S , where each element $s \in S$ has probability $1/|S|$. We define $\mathbb{1}_d = [1, \dots, 1]^\top \in \mathbb{R}^d$. By convention, we have $\binom{n}{m} = 0$ if $n < m$. We use the Kronecker delta function δ_{ij} , where $\delta_{ij} = 1$ if $i = j$ and $\delta_{ij} = 0$ if $i \neq j$. We use $\text{Cov}(\cdot, \cdot)$ to denote the covariance of two random variables. Lastly, $\|\cdot\|_2$ is the ℓ^2 norm, and $\|\cdot\|_\infty$ is the ℓ^∞ norm, both are norms on Euclidean spaces.

2 Preliminaries

In this section, we review the background needed for the rest of the paper. The final layer of a classification network typically outputs a vector of real-valued scores called logits. These logits can be any real number, which makes them difficult to interpret as probabilities that an input belongs to each class. The softmax function maps logits to a probability distribution over classes, i.e., a vector with non-negative entries that sum to one. Such a vector is an element of the unit simplex.

Definition 1 (Unit Simplex) *Let $d \in \mathbb{N}$. The unit simplex in $\mathbb{R}^d$ is denoted by S^{d-1} and is defined as the set of all element-wise non-negative vectors of length d whose entries sum to 1, i.e.,*

$$S^{d-1} = \left\{ p \in \mathbb{R}^d : \sum_{i=1}^d p_i = 1, p_j \geq 0 \text{ for all } j \in [d] \right\}.$$

We will sometimes refer to elements of the simplex as “probability vectors”.

We consider data sets of the form $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ for some $n \in \mathbb{N}$, where x_i is a training input and y_i is its label for each $i \in [n]$. Let $\mathbb{D} = \mathbb{X} \times \mathbb{Y}$ denote the collection of all data sets of interest, where $\mathbb{X}$ is the collection of all training inputs and $\mathbb{Y}$ is the collection of all labels. We do not make any assumptions that $\mathbb{D}$ is a particular type of space (e.g., Euclidean). The goal of differential privacy is to randomize data in a way that makes two “similar” pieces of sensitive data produce outputs that are “approximately indistinguishable”. Given two data sets $\mathcal{D}$ and $\mathcal{D}'$, the notion of “similarity” is formalized by an adjacency relation (Dwork et al., 2014).

Definition 2 (Adjacent Data Sets) *Let $\mathcal{D}, \mathcal{D}' \in \mathbb{D}$. Then $\mathcal{D}$ and $\mathcal{D}'$ are adjacent if one can be obtained from the other by adding or removing a single data point; that is, either $\mathcal{D}' = \mathcal{D} \cup \{(x, y)\}$ or $\mathcal{D} = \mathcal{D}' \cup \{(x, y)\}$ for some data point (x, y) . We write $\text{Adj}(\mathcal{D}, \mathcal{D}') = 1$ if $\mathcal{D}$ and $\mathcal{D}'$ are adjacent and $\text{Adj}(\mathcal{D}, \mathcal{D}') = 0$ otherwise.*

Definition 2 states that two data sets are adjacent if they differ in a single data point. The notion of “approximately indistinguishable” is made precise by the definition of differential privacy itself.

Definition 3 (Differential Privacy, Dwork et al. 2014) Let $\delta \in [0, 1]$ and $\epsilon > 0$ be given. A randomized mechanism $\mathcal{M} : \mathbb{D} \rightarrow S^{d-1}$ is (ϵ, δ) -differentially private if for every pair of data sets $\mathcal{D}, \mathcal{D}' \in \mathbb{D}$ satisfying $\text{Adj}(\mathcal{D}, \mathcal{D}') = 1$ and every measurable output set $\mathcal{T} \subseteq S^{d-1}$, we have

$$\mathbb{P}[\mathcal{M}(\mathcal{D}) \in \mathcal{T}] \leq e^\epsilon \cdot \mathbb{P}[\mathcal{M}(\mathcal{D}') \in \mathcal{T}] + \delta.$$

If $\delta = 0$, then $\mathcal{M}$ is said to be ϵ -differentially private.

In Definition 3, smaller values of ϵ and δ correspond to stronger privacy guarantees. Typical values of ϵ are 0.1 to 10 (Hsu et al., 2014), and typical values of δ are chosen to be much less than $\frac{1}{n}$. An appealing property of differential privacy is that it is immune to post-processing in the sense that arbitrary computations on differentially private data do not weaken its privacy guarantees. Given a data set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n \in \mathbb{D}$, suppose that each entry belongs to an individual and the data set is used to train a model while keeping $\mathcal{D}$ differentially private. Then, Definitions 2 and 3 guarantee that given the trained model or an output it generates, an adversary cannot reliably determine if a specific individual’s data is in $\mathcal{D}$.

We next introduce Rényi differential privacy (RDP), which is a generalization of (ϵ, δ) -differential privacy.

Definition 4 (Rényi Differential Privacy (RDP), Mironov 2017) Fix $\lambda > 1$. A randomized mechanism $\mathcal{M} : \mathbb{D} \rightarrow S^{d-1}$ is $(\lambda, \hat{\epsilon})$ -Rényi differentially private if for all pairs of data sets $\mathcal{D}, \mathcal{D}' \in \mathbb{D}$ satisfying $\text{Adj}(\mathcal{D}, \mathcal{D}') = 1$, we have

$$D_\lambda(\mathcal{M}(\mathcal{D}) \parallel \mathcal{M}(\mathcal{D}')) := \frac{1}{\lambda - 1} \log \mathbb{E}_{y \sim \mathcal{M}(\mathcal{D})} \left[\left(\frac{p_{\mathcal{M}(\mathcal{D})}(y)}{p_{\mathcal{M}(\mathcal{D}')} (y)} \right)^{\lambda-1} \right] \leq \hat{\epsilon},$$

where $p_{\mathcal{M}(\cdot)}$ denotes the probability distribution of the output of $\mathcal{M}(\cdot)$. For $\lambda = 1$, we define $D_1(\mathcal{M}(\mathcal{D}) \parallel \mathcal{M}(\mathcal{D}')) = D_{KL}(\mathcal{M}(\mathcal{D}) \parallel \mathcal{M}(\mathcal{D}'))$.

In Definition 4, smaller values of $\hat{\epsilon}$ and larger values of λ both correspond to stronger privacy guarantees.

At each training step, our method applies a privacy mechanism to provide differential privacy to a random minibatch rather than the full data set. Intuitively, for two adjacent data sets $\mathcal{D}$ and $\mathcal{D}'$, the differing record between the data sets is unlikely to be included in any given minibatch, which amplifies the protections of differential privacy. This idea is formalized by `PoissonSample`, which is often used in private learning settings.

Definition 5 (PoissonSample, Zhu and Wang 2019) Consider a data set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n \in \mathbb{D}$ and select $q \in (0, 1)$. Then, the procedure `PoissonSample` outputs a random subset $\mathcal{D}_B \subseteq \mathcal{D}$ with expected cardinality $q|\mathcal{D}|$ by: (i) sampling $\sigma_i \sim \text{Bernoulli}(q)$ independently for each $i \in [n]$ and (ii) outputting the set $\mathcal{D}_B = \{(x_i, y_i) \in \mathcal{D} : \sigma_i = 1\}$.

Since $\mathbb{E}[|\mathcal{D}_B|] = q|\mathcal{D}|$, we refer to $\mathcal{D}_B$ as a “ q -proportion” of $\mathcal{D}$. When we apply an (ϵ, δ) -differentially private mechanism to a random q -proportion of the data set, privacy amplification ensures the mechanism provides $(\log(1 + q(e^\epsilon - 1)), q\delta)$ -differential privacy to the full data set. It is straightforward to show that this latter privacy protection is $(O(q\epsilon), q\delta)$ -differential privacy, which is stronger than (ϵ, δ) -differential privacy because $q \in (0, 1)$. This

improved privacy guarantee can be combined with the composition of RDP to tightly characterize the privacy loss over many applications of a differential privacy mechanism. As we show in the next section, by composing the privacy loss in the sense of RDP before converting back to standard (ϵ, δ) -differential privacy, we can run our mechanism many times (and hence train a neural network for many epochs) without blowing up the privacy budget ϵ .

3 Proposed Algorithm

In this section, we develop a private training framework that provides privacy to training inputs while leaving labels public. During training, our method perturbs the outputs of a softmax layer by drawing samples from a Dirichlet distribution to enforce differential privacy. Each perturbed output can be interpreted as a vector of perturbed probabilities that a training data point belongs to each class of data. Then these perturbed probabilities and the associated true label are used to compute the loss to be optimized.

We begin by introducing the Dirichlet mechanism. Then we provide the details of our framework and analyze its privacy guarantees. Lastly, we analyze the first and second moments of the perturbed loss to interpret how the training process changes with privacy.

3.1 Dirichlet Mechanism

A mechanism is a map that enforces differential privacy by randomizing functions of sensitive data to guarantee that Definition 3 is satisfied. A mechanism is calibrated using the “sensitivity” of the function being privatized, which is equal to the maximum amount that function’s outputs can differ when it is evaluated on adjacent inputs.

Definition 6 (Sensitivity) *Fix a function $h : \mathbb{X} \rightarrow S^{d-1}$. The ℓ^2 - and ℓ^∞ -sensitivities of h , respectively, are*

$$\Delta_2 = \sup_{x, x' \in \mathbb{X}} \|h(x) - h(x')\|_2$$

and

$$\Delta_\infty = \sup_{x, x' \in \mathbb{X}} \|h(x) - h(x')\|_\infty.$$

In our setting, the sensitive data is the training inputs in the data set $\mathcal{D}$, and the function we privatize is the mapping from neural network inputs to softmax outputs. We privatize at the softmax layer because, regardless of the architecture or current parameters of the neural network, its output lies in the unit simplex, and the compactness of the simplex immediately bounds the neural network’s sensitivity. We next make this observation precise.

Mathematically, a neural network defines a mapping $f_\theta : \mathbb{X} \rightarrow \mathbb{R}^d$ parametrized by θ , where $f_\theta(x)$ denotes the logits associated with an input x . Let $s : \mathbb{R}^d \rightarrow S^{d-1}$ denote the softmax map. At a fixed training step, the current model parameters θ induce a map

$$g_\theta = s \circ f_\theta : \mathbb{X} \rightarrow S^{d-1}$$

so that $g_\theta(x) = s(f_\theta(x))$ maps the training input x to output probabilities. We randomize the outputs of g_θ to privatize its inputs, and therefore we require the sensitivities of g_θ to implement privacy.

Lemma 7 *Let $g_\theta : \mathbb{X} \rightarrow S^{d-1}$ be the model associated with any neural network with a softmax output layer. Then, regardless of the architecture and model parameters, its sensitivities in the sense of Definition 6 are bounded via $\Delta_2^2 \leq 2$ and $\Delta_\infty \leq 1$.*

Proof See Appendix A. ■

We now introduce the Dirichlet mechanism.

Mechanism 8 (Dirichlet Mechanism, Ponnopratt 2023, Section 3.1) *Fix a function $g_\theta : \mathbb{X} \rightarrow S^{d-1}$ parameterized by θ , and fix constants $r > 0$ and $\alpha > 0$. Then, for any input $x \in \mathbb{X}$, the Dirichlet Mechanism $\mathcal{M}_{\text{Dir}}^{(r, \alpha)} : S^{d-1} \rightarrow S^{d-1}$ takes $g_\theta(x)$ as input and outputs $\rho \in S^{d-1}$ sampled from the Dirichlet distribution via*

$$\rho \sim \text{Dir}(rg_\theta(x) + \alpha \mathbb{1}_d),$$

where the pdf of $\text{Dir}(v)$ is $f(p; v) = \frac{1}{B(v)} \prod_{i=1}^d p_i^{v_i-1}$.

The Dirichlet mechanism enforces differential privacy with strength determined by r and α . We will sometimes refer to α as an “offset parameter” and r as a “scaling parameter”.

Lemma 9 (Ponnopratt 2023) *Fix constants $\alpha > 0$ and $r \in (0, \alpha)$, and fix an RDP order $\lambda \in [1, 1 + \frac{\alpha}{r})$. Mechanism 8 enforces $(\lambda, \hat{\epsilon}(\lambda; r))$ -RDP as defined in Definition 4, where*

$$\hat{\epsilon}(\lambda; r) = \lambda r^2 \psi'(\alpha - (\lambda - 1)r).$$

In our analysis, we adapt the RDP bound for PoissonSample from Zhu and Wang (2019), in which the underlying mechanism is fixed while its RDP guarantee is evaluated across different values of λ . Therefore, in our training procedure we fix the Dirichlet mechanism parameters α and r independently of λ .

The privacy guarantee we ultimately seek is for the training inputs within the training set $\mathcal{D}$, i.e., for a point $(x_i, y_i) \in \mathcal{D}$ we seek to privatize x_i . The sensitivities $\Delta_2^2 \leq 2$ and $\Delta_\infty \leq 1$ from Lemma 7 bound how much one forward pass can change between two adjacent training sets, which we will use in Section 3.3 to bound the privacy loss for the entire set of input data.

3.2 Private Training Algorithm

In this subsection, we present a differentially private training framework that privatizes training inputs while leaving labels public. Algorithm 1 outlines the process for training a model with a softmax output layer by minimizing an empirical loss function $\mathcal{L}$. Our complete differentially private training pipeline is as follows. At each training step t , we sample a minibatch $\mathcal{D}_{B_t}$ with sampling rate q by using PoissonSample from Definition 5. Then, for each $i \in B_t$ we compute the output probability vector $g_{\theta_t}(x_i)$ and sample a private probability vector $\tilde{p}^i$ from $\text{Dir}(rg_{\theta_t}(x_i) + \alpha \mathbb{1}_d)$, where $\alpha > 0$ and $r \in (0, \alpha)$ are constants that we choose below. Next, we compute the averaged noisy gradient over the minibatch, which is $\tilde{\xi} = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \mathcal{L}(\tilde{p}^i, y_i)$, where the vector $\tilde{p}^i$ is used in combination with the

true label y_i to compute the NLL¹ loss $\mathcal{L}(\tilde{p}^i, y_i) = -\log \tilde{p}_y^i$. Finally, the algorithm takes a step in the direction $-\tilde{\xi}$. After training for a fixed number of steps T , we output the trained model.

This form of training with the Dirichlet mechanism can potentially require many iterations, and the strength of privacy degrades with each iteration because training data can be reused. If the Dirichlet mechanism is applied to enforce (ϵ, δ) -differential privacy T times, then the standard Composition Theorem (Dwork et al., 2014, Chapter 3.5) shows that the data is protected overall with $(T\epsilon, T\delta)$ -differential privacy. For large T , these protections may become weak. To reduce the rate of weakening of privacy protections, we use *privacy amplification* via PoissonSample (Zhu and Wang, 2019), which we describe in detail in Section 3.3.

Our approach to privacy is optimizer-agnostic in the sense that any optimizer can be used without harming privacy. However, Algorithm 1 is written with stochastic gradient descent (SGD) because SGD is widely used, and that is the algorithm we analyze in Section 3.5.

3.3 Privacy Analysis

To analyze privacy we observe that in Algorithm 1 backpropagation only post-processes private data. Given a minibatch $\mathcal{D}_{B_t}$ at iteration t , lines 3-6 of Algorithm 1 compute the set

$$\mathcal{E}_{B_t} = \{\tilde{p}^i : \tilde{p}^i \sim \text{Dir}(rg_{\theta_t}(x_i) + \alpha \mathbb{1}_d), (x_i, y_i) \in \mathcal{D}_{B_t} \text{ for some } y_i\},$$

and the use of the Dirichlet mechanism ensures that the computation of $\mathcal{E}_{B_t}$ keeps x_i differentially private for each $(x_i, y_i) \in \mathcal{D}_{B_t}$. In Algorithm 1, backpropagation uses only $\mathcal{E}_{B_t}$ and not $\mathcal{D}_{B_t}$, which means that backpropagation post-processes differentially private data. Therefore, the results of backpropagation preserve differential privacy as well. The strength of privacy provided by each iteration of Algorithm 1 is therefore equal to the strength of privacy provided to the training data when computing $\mathcal{E}_{B_t}$ in lines 3-6 of Algorithm 1. We use this fact in the next theorem, which is our main result on the privacy guarantees of Algorithm 1.

Theorem 10 *Fix a sampling rate $q \in (0, 1)$ for PoissonSample. Fix $\alpha > 0$ and $r \in (0, \alpha)$. Then for every integer λ satisfying $2 \leq \lambda < 1 + \frac{\alpha}{r}$, Algorithm 1 is $(\epsilon_o(\lambda, \delta; r), \delta)$ -differentially private with*

$$\begin{aligned} \epsilon_o(\lambda, \delta; r) \leq & \frac{T}{\lambda - 1} \log \left\{ (1 - q)^{\lambda-1} (q\lambda - q + 1) + \binom{\lambda}{2} q^2 (1 - q)^{\lambda-2} e^{\hat{\epsilon}_{\mathcal{M}}(2; r)} \right. \\ & \left. + 3 \sum_{j=3}^{\lambda} \binom{\lambda}{j} (1 - q)^{\lambda-j} q^j e^{(j-1)\hat{\epsilon}_{\mathcal{M}}(j; r)} \right\} + \log(\lambda - 1) - \frac{\log(\delta) + \lambda \log(\lambda)}{\lambda - 1}, \end{aligned}$$

where $\hat{\epsilon}_{\mathcal{M}}(j; r) = jr^2 \psi'(\alpha - (j - 1)r)$ and T is the number of training steps.

Proof See Appendix B. ■

1. We use NLL rather than cross-entropy (CE) loss because our mechanism directly outputs probabilities and typical implementations of the CE loss operate on the logits directly. Without privacy, using a softmax layer with NLL loss is equivalent to using the CE loss on the logits.

Algorithm 1 Training deep neural network classifiers with end-to-end differential privacy

Input: Data set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, NLL loss function $\mathcal{L}$.

Parameters: Learning rate $\gamma > 0$, sampling rate $q \in (0, 1)$, number of training steps $T \in \mathbb{N}$, Dirichlet mechanism parameters $\alpha > 0$ and $r \in (0, \alpha)$.

Initialize θ_0 using the default PyTorch initialization for each layer².

```

1: for  $t = 0, 1, \dots, T - 1$  do
2:   Sample a mini-batch  $\mathcal{D}_{B_t} \subseteq \mathcal{D}$  via PoissonSample from Definition 5 with rate  $q$ .
3:   for each  $(x_i, y_i) \in \mathcal{D}_{B_t}$  do
4:     Compute output probabilities: Set  $g_{\theta_t}(x_i) = \text{softmax}(f_{\theta_t}(x_i))$ .
5:     Perturb output probabilities: Sample  $\tilde{p}^i \sim \text{Dir}(r g_{\theta_t}(x_i) + \alpha \mathbb{1}_d)$ .
6:   end for
7:   Compute gradient:  $\tilde{\xi}_t \leftarrow \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta_t} \mathcal{L}(\tilde{p}^i, y_i)$ .
8:   Update parameters:  $\theta_{t+1} \leftarrow \theta_t - \gamma \tilde{\xi}_t$ .
9: end for
Output:  $\theta_T$ 
    
```

Once the parameters r , λ , and δ are fixed, the privacy parameter $\epsilon_o(\lambda, \delta; r)$ grows as q grows, which is intuitive because a larger sampling rate q means more points are sampled and that there is less benefit from privacy amplification. Therefore, a small value of $q \in (0, 1)$ should be chosen so that the argument of the first log term in Theorem 10 is dominated by its first two terms. Doing so slows the rate of decay of privacy protections as T grows.

The values of r, α, δ, q , and T are always fixed at the beginning of Algorithm 1 so that $\epsilon_o(\lambda, \delta; r)$ is only a function of the RDP order λ . In practice λ is always selected to achieve the minimum value of $\epsilon_o(\lambda, \delta; r)$.

3.4 Selecting Dirichlet Mechanism Parameters

By Theorem 10, Algorithm 1 is (ϵ, δ) -differentially private for some ϵ and δ . Next, for given privacy parameters $\epsilon > 0$ and $\delta > 0$, we seek to select $\alpha > 0$ and $r \in (0, \alpha)$ so that Algorithm 1 is $(\lambda, \hat{\epsilon}(\lambda; r))$ -Rényi differentially private for values of λ and $\hat{\epsilon}(\lambda; r)$ that imply that (ϵ, δ) -differential privacy holds for the chosen values of ϵ and δ .

Our approach is to first fix the offset parameter $\alpha > 0$ and then solve for the largest scaling parameter $r > 0$ that ensures the enforcement of (ϵ, δ) -differential privacy. For any $r > 0$ the admissible RDP orders are

$$\Lambda(r) = \left\{ \lambda \in \mathbb{N} \mid 2 \leq \lambda < 1 + \frac{\alpha}{r} \right\},$$

and we define $h(r) = \min_{\lambda \in \Lambda(r)} \epsilon_o(\lambda, \delta; r)$. Theorem 10 implies that running Algorithm 1 for T steps is $(h(r), \delta)$ -differentially private. To find the largest r that the privacy budget allows we compute

$$r^* = \sup \{ r \in (0, \alpha) \mid h(r) \leq \epsilon \}. \quad (1)$$

2. For network parameters θ_0 , weights and biases of linear and convolutional layers are drawn from the uniform distribution on the interval $(-\frac{1}{\sqrt{k}}, \frac{1}{\sqrt{k}})$, where k is the number of input features per neuron, normalization scaling factors are set to 1, and normalization biases are set to 0.

Since $h(r)$ is increasing in r and $\lim_{r \rightarrow \alpha} h(r) = \infty$, we see that for any $\epsilon > \lim_{r \rightarrow 0} h(r)$ there is an $r < \alpha$ such that $h(r) \leq \epsilon$. Such an r can be found, for example, via root-finding on the equation $h(r) = \epsilon$. In our experiments in Section 4 we solve this root-finding problem with Brent’s method (Brent, 1971).

3.5 Analytical Characterization of Algorithm 1

We now characterize how the choices of the mechanism parameters (r, α) affect the optimization steps in Algorithm 1. Although Algorithm 1 generates a sequence of parameters $\{\theta_t\}_{t=0}^T$, in this subsection we analyze the effect of privacy for arbitrary network parameters θ . All results in this subsection apply at any training step with index $t \in [T] \cup \{0\}$ by setting $\theta = \theta_t$. Since r is automatically determined by (1) in terms of a fixed α and fixed privacy budget (ϵ, δ) , the behavior of Algorithm 1 is governed by α and the learning rate γ , and our analysis is in terms of these parameters. The main result of this section is that the perturbations due to the Dirichlet mechanism do not change the direction of the expected gradient of the per-sample loss of Algorithm 1. Instead, these perturbations result in an expected private gradient that is equal to a rescaled version of the non-private cross-entropy gradient, and this scaling is captured by a data-dependent scale factor.

Consider fixed network parameters θ and a training example $(x, y) \in \mathcal{D}$. Let $z = f_\theta(x)$ be the logits associated with x . Recall that $s(z) = \text{softmax}(z)$ and $g_\theta(x) = s(f_\theta(x)) = s(z)$. We use $s_j(z)$ to denote the probability assigned to class $j \in [d]$ and $s_y(z)$ to denote the probability assigned to the true class y .

The cross-entropy loss without privacy is $\mathcal{L}_{\text{CE}}(z) = -\log s_y(z)$, which has gradient $\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_j}(z) = s_j(z) - \delta_{yj}$ for $j \in [d]$. Algorithm 1 perturbs $s(z)$ by drawing a Dirichlet sample and substituting it into the negative log-likelihood loss to compute $\mathcal{L}(\tilde{p}(z), y) = -\log \tilde{p}_y$, where $\tilde{p}(z) \sim \text{Dir}(rs(z) + \alpha \mathbb{1}_d)$ and $r, \alpha > 0$ are the mechanism parameters.

To characterize training using Algorithm 1, we analyze the first two moments of the stochastic loss. The expected loss

$$F(z) := \mathbb{E}_{\tilde{p}}[\mathcal{L}(\tilde{p}(z), y)] \quad (2)$$

and the expected gradient $\mathbb{E}_{\tilde{p}} \left[\frac{\partial}{\partial z_j} \mathcal{L}(\tilde{p}(z), y) \right]$ quantify a training data point’s expected influence on the gradient in Line 7 of Algorithm 1, while $\text{Var}_{\tilde{p}}[\mathcal{L}(\tilde{p}(z), y)]$ measures the variability of the loss around its mean.

Theorem 11 *Fix a data set $\mathcal{D} \in \mathbb{D}$ and consider training a model using Algorithm 1 with $r, \alpha > 0$. Consider a single training example $(x, y) \in \mathcal{D}$ with logits $z = f_\theta(x)$ and $s(z) = \text{softmax}(z)$. Define the scalar attenuation factor*

$$\kappa(z; r, \alpha) := r s_y(z) \psi'(rs_y(z) + \alpha). \quad (3)$$

Then for every $z \in \mathbb{R}^d$ and every $j \in [d]$, for F in (2) we have

$$F(z) = \psi(r + d\alpha) - \psi(rs_y(z) + \alpha) \quad (4)$$

$$\begin{aligned} \mathbb{E}_{\tilde{p}} \left[\frac{\partial}{\partial z_j} \mathcal{L}(\tilde{p}(z), y) \right] &= \kappa(z; r, \alpha) \frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_j}(z) \\ \text{Var}_{\tilde{p}}[\mathcal{L}(\tilde{p}(z), y)] &= \psi'(rs_y(z) + \alpha) - \psi'(r + d\alpha). \end{aligned} \quad (5)$$

Proof See Appendix C. ■

The last three equations in Theorem 11 have simple interpretations. The first shows that the expected private loss depends on the logits only through the predicted true class probability $s_y(z)$. The second is the key identity, and it shows that, on a single data point (x, y) , the expected gradient of the loss with respect to the logits is equal to the clean, non-private cross-entropy gradient multiplied by the scaling factor $\kappa(z; r, \alpha)$. Thus, on a per-sample basis, our privacy implementation preserves the gradient’s direction and only rescales its magnitude. The third equation quantifies the randomness introduced by the Dirichlet mechanism through the variance of the stochastic loss.

3.6 Selecting Learning Rates

Theorem 11 can be used to calibrate the learning rate of Algorithm 1. In Line 8 of Algorithm 1, the parameters of the neural network are updated with the average gradient over a batch. Using Theorem 11, we compute the expected gradient evaluated on the output of the Dirichlet mechanism in terms of the logits z produced by a single training sample x . This gradient is scaled by $\kappa(z; r, \alpha)$ from (3), and the corresponding expected SGD update is

$$\mathbb{E}_{\tilde{p}}[-\gamma \nabla \mathcal{L}(\tilde{p}(z), y)] = -\gamma \kappa(z; r, \alpha) \nabla \mathcal{L}_{\text{CE}}(z),$$

where $\nabla \mathcal{L}_{\text{CE}}(z)$ is the gradient of the non-private cross-entropy loss evaluated on the logits z of the training example x .

Therefore, on average, a gradient step with privacy implemented with learning rate γ behaves the same as a gradient step without privacy with learning rate

$$\gamma_{\text{eff}} = \gamma \kappa(z; r, \alpha), \tag{6}$$

which we call the *effective learning rate* of such an implementation. The attenuation factor $\kappa(z; r, \alpha)$ is sample-dependent through its dependence on z , and we of course cannot have a sample-dependent learning rate. However, the following empirical results show that if we simply use the value $s_y(z) = 0.9$ for all examples and compare training runs using different values of α but the same value of $\gamma_{\text{eff}} = \gamma \kappa(z; r, \alpha)$, then we achieve similar performance across those values of α . These results imply that for varying values of the parameter α , the learning rate γ of Algorithm 1 can be tuned to counter the effects of privacy and achieve a nearly constant level of accuracy.

Figure 2 provides empirical results for the preceding analysis. We consider $\alpha \in \{0.1, 0.2, 0.3, 0.5, 1, 2, 3, 5, 10\}$, and for each α we first find r^* using (1) with privacy parameters $\epsilon = 1$ and $\delta = 10^{-5}$. For each α we train a neural network using varying values of γ , specified in Appendix C.2, and for each value we train 10 instances of ResNet-18 on CIFAR10 and report the mean test set accuracy in the left plot of Figure 2. Still using $s_y(z) = 0.9$ for all examples, we then compute the test accuracy values for varying values of α and γ , and we plot them against the effective learning rate $\gamma_{\text{eff}} = \gamma \kappa(z; r, \alpha)$ on the right-hand side of Figure 2. The different values of α produce curves that are quite similar, and this behavior shows that at a fixed privacy budget the dependence of performance on (α, γ) is largely explained by the change in the average update scale, namely $\kappa(z; r, \alpha)$.

In Appendix C.1 we analyze the asymptotic behavior of $\kappa(z; r, \alpha)$ and $\text{Var}_{\tilde{p}}[\mathcal{L}(\tilde{p}(z), y)]$ as functions of α . At small values of α , the value of $\kappa(z; r, \alpha)$ decays like α^{-1} , while the variance

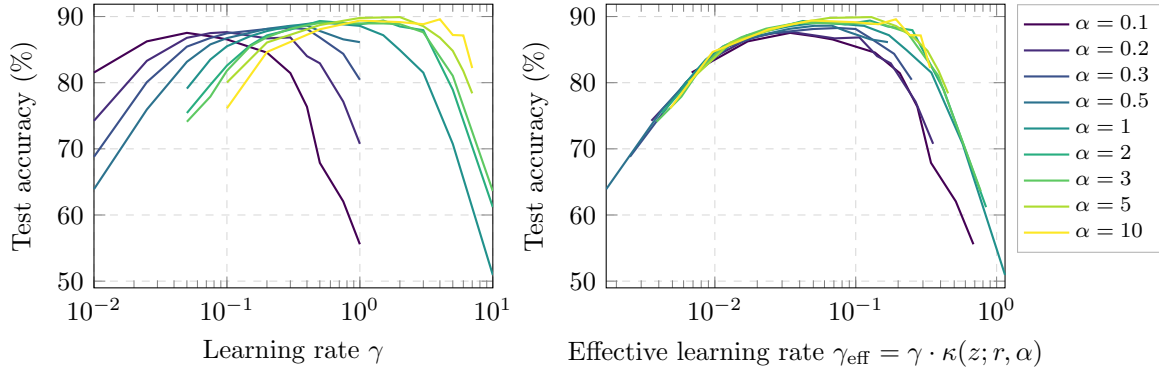


Figure 2: Mean test accuracy of ResNet-18 on CIFAR10 under a fixed privacy budget ($\epsilon = 1, \delta = 10^{-5}$) for $\alpha \in [0.1, 10]$. The left plot shows the mean test accuracy as a function of the learning rate γ . The right plot shows the same data plotted against the effective learning rate γ_{eff} from (6), which produces overlapping curves and shows that much of accuracy’s dependence on (α, γ) is explained by the update scale reparameterization due to $\kappa(z; r, \alpha)$.

decays like α^{-2} . At large α , the expected update scale saturates, while the variance decays like α^{-1} . Thus, increasing α reduces the randomness of the Dirichlet mechanism faster than it reduces the average update size, which allows for runs with larger values of α to use larger learning rates γ .

4 Evaluation

In this section we evaluate the proposed framework on standard benchmark data sets that have been used in previous works on private machine learning³.

4.1 Experimental Setup

4.1.1 DATA SETS AND MODELS

We evaluate Algorithm 1 on CIFAR10 (Krizhevsky et al., 2009), MNIST (LeCun et al., 1998), DermaMNIST from MedMNIST (Yang et al., 2023, 2021), FashionMNIST (Xiao et al., 2017), and Street View House Numbers (SVHN) (Netzer et al., 2011). For CIFAR10, we use ResNet-18 (He et al., 2016), which was also used in Monir and Ghinita (2024) when privatizing training inputs while leaving labels public. For MNIST and FashionMNIST (Xiao et al., 2017), we use a simplified Inception model following Szegedy et al. (2015). For DermaMNIST, we use ResNet-9 following Hölzl et al. (2022). For SVHN, we use ResNet-18. We provide results on CIFAR10, MNIST, DermaMNIST, FashionMNIST and SVHN in Section 4.2.

3. Our code is available at <https://github.com/Rhyme0730/Dirichlet-mechanism-for-private-classification>.

4.1.2 IMPLEMENTATION DETAILS

For all data sets, we use the parameters $\alpha = 3.0$ and $\delta = 10^{-5}$. We train with standard SGD, NLL loss, and PoissonSample for all data sets. We use learning rates $\gamma = 0.05$ for DermaMNIST and $\gamma = 0.1$ for all other data sets. Algorithm 1 is run for $E = qT$ epochs on each data set, where q is the parameter used in PoissonSample. For CIFAR10, we apply standard data augmentations, including random cropping and random horizontal flipping, and training is done with sampling rate $q = 250/50000 = 0.005$ for $T = 20,000$ steps (100 epochs) to ensure a fair comparison with previous work in (Monir and Ghinita, 2024). On DermaMNIST, we use sampling rate $q = 70/7007 \approx 0.01$ and train for $T = 6006$ steps (60 epochs). On MNIST and FashionMNIST, we use sampling rate $q = 250/60000 \approx 0.0042$ and train for $T = 9600$ steps (40 epochs). On SVHN, we use sampling rate $q = 250/73257 \approx 0.0034$ and train for $T = 11,722$ steps (40 epochs). The parameter r^* is computed for each data set using (1) based on the training configuration above and the target value of the privacy parameter ϵ . On FashionMNIST and SVHN, we use Opacus (Yousefpour et al., 2021) for our DP-SGD benchmark with maximum gradient clipping norm 1.0. We use the SGD optimizer with learning rate 0.1 and no data augmentation with ResNet-18 as a baseline. We report the value of r^* we use in each experiment in Appendix D.2. In the rest of this section, the mean accuracy and standard deviation for all data sets are reported across 5 independent runs with different random seeds for each value of ϵ . We report results for different sampling rates in Appendix F and for different optimizers in Appendix G. All reported results are obtained from models trained from scratch⁴, not through fine-tuning existing models. We use PyTorch (Paszke et al., 2019) for our implementation and train all the models on Nvidia Quadro RTX6000 GPUs.

4.2 Evaluation of the Proposed Framework

We report the results of the proposed framework in Tables 1, 2, and 3 for the CIFAR10, MNIST, and DermaMNIST data sets, respectively. We reiterate that DP-SGD privatizes both training inputs and labels, while the current paper and Monir and Ghinita (2024) only privatize training inputs. Therefore, the privacy protections of DP-SGD are strictly stronger than the protections provided by Algorithm 1, though we still compare to DP-SGD because it is widely used in private learning.

4.2.1 CIFAR10 RESULTS

Table 1 compares Algorithm 1 to DP-SGD (De et al., 2022) using the best results that DP-SGD produces on CIFAR10 from scratch with Wide-ResNet (WRN-40-4), large-batch training, and augmentation multiplicity. According to Zagoruyko and Komodakis (2016), the WRN-40-4 model obtains $> 95\%$ accuracy on CIFAR10 when trained non-privately. Algorithm 1 under the configuration in Section 4.1 obtains higher performance than the

4. We run experiments for $\epsilon = \infty$ using PoissonSample but without privacy. The accuracy degradation from the non-private baseline at $\epsilon = \infty$ is caused by using PoissonSample and not adding momentum or weight decay to the SGD optimizer, which we choose to do in order to have a meaningful baseline for assessing the performance of Algorithm 1. Results marked with “†” indicate non-private ($\epsilon = \infty$) baselines that were omitted from (Monir and Ghinita, 2024). We reproduced these values using the configurations described in the references corresponding to each method.

Method	Model	$\epsilon = 1$	$\epsilon = 2$	$\epsilon = 4$	$\epsilon = 6$	$\epsilon = 8$	$\epsilon = \infty$
DP-SGD (De et al., 2022)	WRN-40-4	56.4	65.9	73.5	78.8	81.4	—
Monir and Ghinita (2024)	ResNet-18	—	—	78.37	79.0 _($\epsilon=4.46$)	—	92.35 [†]
This work	ResNet-18	82.96 ± 1.46	86.34 ± 0.34	88.17 ± 0.33	88.54 ± 0.54	88.61 ± 0.34	92.35

Table 1: Test accuracy (%) on CIFAR10 under $\delta = 10^{-5}$ across 5 independent runs for each ϵ .

WRN-40-4 configurations in (De et al., 2022) for all values of $\epsilon \in [1, 8]$. Notably, when $\epsilon = 1$, the test accuracy of DP-SGD is 56.4%, while the test accuracy of Algorithm 1 is 82.96%, which is a 60.92% reduction in test error rate.

We also compare to the closest prior work, which is Monir and Ghinita (2024), using the same model (ResNet-18), the same optimizer, and the same sampling rate q of PoissonSample. Since the learning rate γ is not reported in Monir and Ghinita (2024), we use $\gamma = 0.1$ in training CIFAR10 for Algorithm 1. We see that Algorithm 1 has a 45.3% lower error rate than Monir and Ghinita (2024) when $\epsilon = 4$. Since the accuracy at the strongest privacy level for Algorithm 1 ($\epsilon = 1$) already exceeds the reported accuracy at the weakest privacy level in Monir and Ghinita (2024) ($\epsilon = 4.46$), we omit the accuracy of Algorithm 1 at $\epsilon = 4.46$ from Table 1.

4.2.2 MNIST RESULTS

As shown in Table 2, Algorithm 1 has higher accuracy than all DP-SGD baselines and the method of Monir and Ghinita (2024) across all tested values of ϵ . We applied Algorithm 1 to a simplified Inception model (Szegedy et al., 2015), which is a type of convolutional neural network (CNN). In Table 2, we see that the accuracy of the Inception model at $\epsilon = 0.5$ closely matches the non-private accuracy of the Inception model, demonstrating that Algorithm 1 preserves high utility even under strong privacy protections. Note that Papernot et al. (2021) and Cheng et al. (2022) only report results at $\epsilon = 2.93$ and $\epsilon = 3$, respectively, and we compare the accuracy of those methods to the accuracy of Algorithm 1 at nearby ϵ

Method	Model	$\epsilon = 0.5$	$\epsilon = 1$	$\epsilon = 2$	$\epsilon = 4$	$\epsilon = \infty$
DP-SGD	MLP	90.0	—	95.0	—	98.3
Papernot et al. (2021)	CNN	—	—	—	98.1 _($\epsilon=2.93$)	99.0
Cheng et al. (2022)	DPNASNet	—	—	—	98.57 _($\epsilon=3$)	—
Monir and Ghinita (2024)	CNN	—	98.0	—	—	99.4 [†]
This work	Inception	99.28 ± 0.072	99.37 ± 0.059	99.40 ± 0.058	99.44 ± 0.038	99.51

Table 2: Test accuracy (%) on MNIST under $\delta = 10^{-5}$ across 5 independent runs for each value of ϵ .

Method	Model	$\epsilon = 1$	$\epsilon = 4$	$\epsilon = 7.42$	$\epsilon = \infty$
DP-SGD (Tang et al., 2023)	ResNet-9	68.34	71.08	72.58	76.16
This work	ResNet-9	74.32 ± 1.84	75.74 ± 0.74	76.31 ± 1.15	78.60

Table 3: Test accuracy (%) on DermaMNIST under $\delta = 10^{-5}$ across 5 independent runs for each ϵ .

values. We also compare to DP-SGD (Abadi et al., 2016) which uses a multilayer perceptron (MLP) model. Even at the strongest privacy level of $\epsilon = 0.5$, Algorithm 1 produces higher accuracy than Monir and Ghinita (2024) at $\epsilon = 1$. Also, the method in Monir and Ghinita (2024) requires a specific funnel-shaped CNN designed to reduce the noise injected during training, while Algorithm 1 can be applied to any neural network architecture that has a softmax output layer. The improvement in test accuracy of Algorithm 1 over Monir and Ghinita (2024) is due to two factors: (i) we inject randomness only at the softmax layer rather than at layers throughout the network, and (ii) under the same privacy budget, the Dirichlet mechanism introduces randomness with lower variance than the Gaussian mechanism from Monir and Ghinita (2024).

4.2.3 DERMAMNIST RESULTS

We have shown that Algorithm 1 outperforms previous work on the standard computer vision benchmarks of CIFAR10 and MNIST, and we now extend our evaluation to medical images, where the privacy of input data is critical. For training, we applied Algorithm 1 to ResNet-9, which was also used in (Hözl et al., 2022). There is limited previous work that evaluates the accuracy of private machine learning models on sensitive medical data such as DermaMNIST (Yang et al., 2021, 2023). Evaluating privately learned models on this data set directly highlights the practical significance of Algorithm 1, which privatizes inputs that are medical images to ensure privacy for patients. In Table 3 we see that Algorithm 1 outperforms the benchmark DP-SGD method (Tang et al., 2023) from $\epsilon = 1$ to $\epsilon = 7.42$ when using the same architecture⁵. Furthermore, the gap between our accuracy at $\epsilon = 1$ and the non-private baseline is less than 5%, demonstrating that Algorithm 1 provides strong privacy protections while maintaining high model accuracy. Due to DermaMNIST’s train/validation/test partitioning, we also report the validation accuracy in Appendix E.

4.2.4 FASHIONMNIST AND SVHN RESULTS

As shown in Figure 3, Algorithm 1 has higher accuracy than DP-SGD across all tested values of ϵ . Notably, on FashionMNIST at $\epsilon = 4.0$, Algorithm 1 achieves a 48.93% reduction in test error rate relative to DP-SGD. Algorithm 1 also outperforms Tempered Sigmoid (Papernot et al., 2021), which reports the highest accuracy among existing methods for private learning. On SVHN at $\epsilon = 1.0$, Algorithm 1 achieves a 35.77% reduction in test error rate

5. The accuracy gap at $\epsilon = \infty$ between DP-SGD and Algorithm 1 arises from DP-SGD’s use of group normalization (required for DP compatibility) versus batch normalization used in our work.

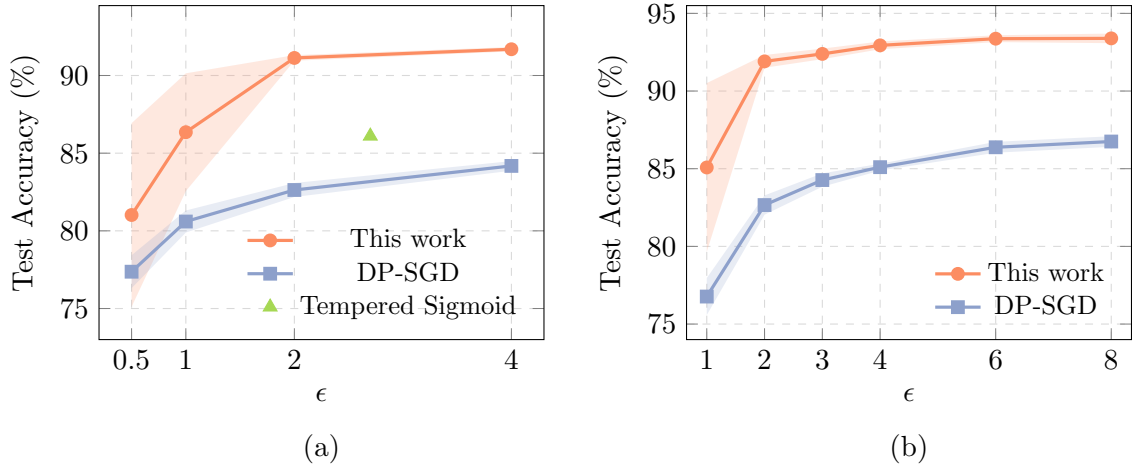


Figure 3: Test accuracy (%) under $\delta = 10^{-5}$ across 5 independent runs for each ϵ on (a) FashionMNIST and (b) SVHN. Shaded bands show one standard deviation for each ϵ .

Method	Model	$\epsilon = 0.5$	$\epsilon = 1$	$\epsilon = 2$	$\epsilon = 4$	$\epsilon = \infty$
Tempered Sigmoid	CNN	—	—	86.1 _($\epsilon=2.7$)	—	89.4
DP-SGD	Inception	77.37 ± 1.06	80.60 ± 0.71	82.63 ± 0.45	84.18 ± 0.31	93.13
This work	Inception	81.02 ± 5.91	86.35 ± 3.79	91.13 ± 0.18	91.69 ± 0.14	93.13

Table 4: Test accuracy (%) on FashionMNIST under $\delta = 10^{-5}$ across 5 independent runs for each ϵ .

relative to DP-SGD. The full results are shown in Table 4 for FashionMNIST and Table 5 for SVHN.

4.2.5 COMPUTATIONAL COST

Our private training framework can be split into two stages. First, we compute r^* using (1) for a fixed DP budget (ϵ, δ) and a fixed value of α , which usually takes less than one minute for each $\epsilon \in [1, 8]$. Once r^* is computed, we follow the traditional image training process with an additional step of perturbing output probabilities using the Dirichlet mechanism at each training step, for which the additional time complexity is $O(dq|\mathcal{D}|T)$ for the whole training process, where $q|\mathcal{D}|$ is the expected mini-batch size at each iteration of Algorithm 1 and d is the number of classes. Compared to previous work (Monir and Ghinita, 2024) that follows the DP-SGD approach for adding noise to model layer outputs and using per-sample clipping, the training process of Algorithm 1 is significantly faster. When privately training

Method	Model	$\epsilon=1$	$\epsilon=2$	$\epsilon=3$	$\epsilon=4$	$\epsilon=6$	$\epsilon=8$	$\epsilon=\infty$
DP-SGD	ResNet-18	76.77 ± 1.19	82.66 ± 0.62	84.27 ± 0.43	85.10 ± 0.22	86.38 ± 0.35	86.75 ± 0.34	94.58
This work	ResNet-18	85.08 ± 5.43	91.91 ± 0.41	92.39 ± 0.34	92.94 ± 0.25	93.37 ± 0.22	93.39 ± 0.31	94.58

Table 5: Test accuracy (%) on SVHN under $\delta = 10^{-5}$ across 5 independent runs for each ϵ .

ResNet-18 on CIFAR10, training an epoch (200 steps) for Algorithm 1 consumes ~ 30 seconds, while for DP-SGD it consumes ~ 60 seconds. The main reason Algorithm 1’s training speed is faster is because per-sample clipping is unnecessary for it.

5 Discussion

Monir and Ghinita (2024) show that injecting calibrated Gaussian noise into the penultimate layer of a neural network can privatize inputs by randomizing features while keeping labels public, and that work demonstrates that this approach to privacy produces higher accuracy than DP-SGD. However, the variance of privacy noise in Monir and Ghinita (2024) scales with the width of the penultimate layer, which degrades utility for larger models. In contrast, the variance of the Dirichlet mechanism scales with the number of classes in the data set, yielding substantially better utility under the same privacy guarantee.

A second key difference is that Algorithm 1 eliminates the need for per-sample gradient clipping, which Monir and Ghinita (2024) use. Clipping degrades utility and slows training, since each sample’s gradient must be clipped individually. Comparing the improvements between Algorithm 1 and Monir and Ghinita (2024) shows two contributions of our work: (i) the use of the Dirichlet mechanism instead of the Gaussian mechanism and (ii) the choice of layer at which privacy is introduced. At $\epsilon = 4.0$, we observe a 45.31% lower error rate in test accuracy on CIFAR10 relative to the method in Monir and Ghinita (2024), while using the same architecture (ResNet-18). And the accuracy of Algorithm 1 at the strongest privacy level we test ($\epsilon = 1.0$) exceeds the accuracy of the method of Monir and Ghinita (2024) at the weakest privacy level they test ($\epsilon = 4.46$).

While DP-SGD (Abadi et al., 2016) provides a stronger privacy guarantee since it privatizes both training inputs and labels, it has worse performance compared to Algorithm 1. Algorithm 1 only provides privacy to training inputs, which allows it to introduce less randomness into training and produce models with higher accuracy. This difference allows users to choose which part of their data requires privacy protections, and Algorithm 1 allows users to avoid unnecessary accuracy loss from privatizing labels that do not need privacy protections. Algorithm 1 is therefore complementary to DP-SGD because it provides different privacy protections and serves different use cases than DP-SGD.

6 Conclusion

In this work, we introduced a novel private training procedure that privatizes input data while keeping labels public when training deep neural networks. We showed that the Dirichlet mechanism for differential privacy can be implemented at a softmax layer, with the resulting randomized probabilities used for computing the loss and backpropagation. The overall framework can be broken into two stages: (i) using an upper bound on the privacy budget to determine the parameters for the Dirichlet mechanism that produce the desired variance of privatized data, and (ii) implementing the Dirichlet mechanism at the softmax layer at each training step. Empirical evaluations on CIFAR10, MNIST, DermaMNIST, Fashion MNIST and SVHN demonstrate that our framework substantially outperforms the prior state of the art by improving test accuracy from 78.37% to 88.17% at $\epsilon = 4$, and retaining meaningful utility even under strong privacy at $\epsilon = 1$. Furthermore, the improvements of our approach persist across different image data sets, model architectures, and optimizers. Future work includes the development of additional parameter selection rules to boost accuracy, as well as extending the private training framework to other machine learning tasks such as language, speech, and financial modeling.

Acknowledgments and Disclosure of Funding

This work was supported by NSF under CAREER grant 2422260 and Graduate Research Fellowship grant DGE-2039655, AFOSR under grant FA9550-19-1-0169, and ONR under grant N00014-24-1-2432. The authors declare no competing interests.

Appendix A. Proof of Lemma 7

For any inputs $x, x' \in \mathbb{X}$ and any adjacent data sets $\mathcal{D}$ and $\mathcal{D}'$, both $g_\theta(x)$ and $g_\theta(x')$ lie in the simplex S^{d-1} because a softmax layer always outputs a probability vector. We define $a = g_\theta(x)$ and $b = g_\theta(x')$, and for $k \in [d]$ we use $a_k, b_k \geq 0$ to denote the k^{th} entries of a and b , respectively. Then $\sum_{k=1}^d a_k = \sum_{k=1}^d b_k = 1$. For the ℓ^2 -sensitivity, we have

$$\|a - b\|_2^2 = \sum_{k=1}^d (a_k - b_k)^2 = \sum_{k=1}^d a_k^2 + \sum_{k=1}^d b_k^2 - 2 \sum_{k=1}^d a_k b_k.$$

Then, since $a_k, b_k \in [0, 1]$ we have $\sum_{k=1}^d a_k^2 \leq \sum_{k=1}^d a_k = 1$, along with $\sum_{k=1}^d b_k^2 \leq \sum_{k=1}^d b_k = 1$, and $\sum_{k=1}^d a_k b_k \geq 0$. Hence

$$\Delta_2^2 = \sup_{x, x' \in \mathbb{X}} \|g_\theta(x) - g_\theta(x')\|_2^2 \leq 1 + 1 - 0 \leq 2.$$

For the ℓ_∞ -sensitivity, we have

$$\|a - b\|_\infty \leq \max_{k \in [d]} \{a_k, b_k\} \leq 1,$$

and therefore

$$\Delta_\infty = \sup_{x, x' \in \mathbb{X}} \|g_\theta(x) - g_\theta(x')\|_\infty \leq 1.$$

Appendix B. Proof of Theorem 10

We use the lemmas below in proving Theorem 10.

B.1 Supporting Lemmas

Lemma 12 (Composition of RDP mechanisms, Mironov 2017) *Fix $\lambda \geq 1$ and $k \in \mathbb{N}$ privacy mechanisms $\mathcal{M}_1, \dots, \mathcal{M}_k$, where mechanism i is $(\lambda, \hat{\epsilon}_{\mathcal{M}_i}(\lambda))$ -RDP for all $i \in [k]$. Let $\mathcal{M} = (\mathcal{M}_1, \dots, \mathcal{M}_k)$, where, for all $i \in [k] \setminus \{1\}$, $\mathcal{M}_i$ can depend on the outputs of $\mathcal{M}_1, \dots, \mathcal{M}_{i-1}$. Then $\mathcal{M}$ is $(\lambda, \hat{\epsilon}_{\mathcal{M}}(\lambda))$ -RDP with $\hat{\epsilon}_{\mathcal{M}}(\lambda) = \sum_{i=1}^k \hat{\epsilon}_{\mathcal{M}_i}(\lambda)$.*

Lemma 13 (RDP to DP conversion, Canonne et al. 2020) *Let $\epsilon, \delta > 0$. If $\mathcal{M} : \mathbb{D} \rightarrow S^{d-1}$ is $(\lambda, \hat{\epsilon})$ -Renyi differentially private in the sense of Definition 4, then it is (ϵ, δ) -differentially private in the sense of Definition 3, with*

$$\epsilon = \hat{\epsilon} + \log(\lambda - 1) - \frac{\log(\delta) + \lambda \log(\lambda)}{\lambda - 1}.$$

B.2 Proof

Each iteration of Algorithm 1 applies Mechanism 8 to a Poisson subsampled minibatch of $\mathcal{D}$. By Lemma 9, for any $\lambda \in [1, 1 + \frac{\alpha}{r})$ a single application of the Dirichlet mechanism enforces $(\lambda, \hat{\epsilon}_{\mathcal{M}}(\lambda; r))$ -Renyi differential privacy with $\hat{\epsilon}_{\mathcal{M}}(\lambda; r) = \lambda r^2 \psi'(\alpha - (\lambda - 1)r)$. In (Ponnoprat, 2023), α is set to be a function of r and the RDP order λ . In our analysis, we adapt the moments accountant from (Zhu and Wang, 2019), in which the underlying mechanism must

be fixed while its RDP protections are evaluated across different values of λ and $\hat{\epsilon}_{\mathcal{M}}(\lambda; r)$. Therefore, in our training procedure we fix the Dirichlet mechanism parameters α and r independently of λ .

Since each minibatch is drawn using Poisson subsampling with rate q , the per-step RDP protections are amplified. It follows from (Zhu and Wang, 2019, Theorem 5) that for any integer value of $\lambda \in [2, 1 + \frac{\alpha}{r})$, we have

$$\begin{aligned} \hat{\epsilon}_{sub}(\lambda; r) &\leq \frac{1}{\lambda - 1} \log \left\{ (1 - q)^{\lambda - 1} (q\lambda - q + 1) \right. \\ &\quad \left. + \binom{\lambda}{2} q^2 (1 - q)^{\lambda - 2} e^{\hat{\epsilon}_{\mathcal{M}}(2; r)} + 3 \sum_{j=3}^{\lambda} \binom{\lambda}{j} (1 - q)^{\lambda - j} q^j e^{(j-1)\hat{\epsilon}_{\mathcal{M}}(j; r)} \right\}, \end{aligned}$$

where $(\lambda, \hat{\epsilon}_{sub}(\lambda; r))$ is the strength of RDP protection provided to the minibatch at a single training step.

Over T training steps, the overall RDP guarantees compose according to Lemma 12, and the training data is protected with $(\lambda, \hat{\epsilon}_o(\lambda; r))$ -RDP, where

$$\begin{aligned} \hat{\epsilon}_o(\lambda; r) &\leq \frac{T}{\lambda - 1} \log \left\{ (1 - q)^{\lambda - 1} (q\lambda - q + 1) \right. \\ &\quad \left. + \binom{\lambda}{2} q^2 (1 - q)^{\lambda - 2} e^{\hat{\epsilon}_{\mathcal{M}}(2; r)} + 3 \sum_{j=3}^{\lambda} \binom{\lambda}{j} (1 - q)^{\lambda - j} q^j e^{(j-1)\hat{\epsilon}_{\mathcal{M}}(j; r)} \right\}. \end{aligned}$$

Lastly, using Lemma 13 for a fixed $\delta > 0$ gives

$$\begin{aligned} \epsilon_o(\lambda, \delta; r) &\leq \frac{T}{\lambda - 1} \log \left\{ (1 - q)^{\lambda - 1} (q\lambda - q + 1) + \binom{\lambda}{2} q^2 (1 - q)^{\lambda - 2} e^{\hat{\epsilon}_{\mathcal{M}}(2; r)} \right. \\ &\quad \left. + 3 \sum_{j=3}^{\lambda} \binom{\lambda}{j} (1 - q)^{\lambda - j} q^j e^{(j-1)\hat{\epsilon}_{\mathcal{M}}(j; r)} \right\} + \log(\lambda - 1) - \frac{\log(\delta) + \lambda \log(\lambda)}{\lambda - 1} \end{aligned}$$

for any integer value of $\lambda \in [2, 1 + \frac{\alpha}{r})$.

Appendix C. Proof of Theorem 11

We first prove

$$F(z) = \mathbb{E}_{\tilde{p}}[\mathcal{L}(\tilde{p}(z), y)] = \psi(r + d\alpha) - \psi(rs_y(z) + \alpha) \quad (7)$$

and

$$\text{Var}_{\tilde{p}}[\mathcal{L}(\tilde{p}(z), y)] = \psi'(rs_y(z) + \alpha) - \psi'(r + d\alpha). \quad (8)$$

The derivation of these equations uses properties of the exponential family of distributions:

Definition 14 (Wainwright and Jordan 2008, Proposition 3.1) *The exponential family of distributions contains densities of the form*

$$p(x; \vartheta) = \exp \left(\sum_{i=1}^d \vartheta_i \phi_i(x) - A(\vartheta) \right) h(x),$$

where $x \in \mathbb{R}^n$ denotes a generic realization of the random variable being modeled and $\vartheta \in \mathbb{R}^d$ is the parameter indexing the distribution. For $i, j \in [d]$ these distributions satisfy

$$\frac{\partial A(\vartheta)}{\partial \vartheta_i} = \mathbb{E}[\phi_i(X)] \quad (9)$$

$$\frac{\partial^2 A(\vartheta)}{\partial \vartheta_i \partial \vartheta_j} = \text{Cov}(\phi_i(X), \phi_j(X)). \quad (10)$$

Let $z \in \mathbb{R}^d$ be the network logits for a single example x with true label $y \in [d]$. Let $\eta(z) = rs(z) + \alpha \mathbb{1}_d$, with

$$\eta_y(z) = rs_y(z) + \alpha, \quad \tau = \sum_{j=1}^d \eta_j(z) = r + d\alpha. \quad (11)$$

The probability density function used in Mechanism 8 takes the form

$$f(\tilde{p}; \eta(z)) = \frac{1}{B(\eta(z))} \prod_{j=1}^d \tilde{p}_j^{\eta_j(z)-1}, \quad \tilde{p} \in S^{d-1}.$$

Writing this density as

$$f(\tilde{p}; \eta(z)) = \exp \left(-\log B(\eta(z)) + \sum_{j=1}^d (\eta_j(z) - 1) \log \tilde{p}_j \right),$$

we observe that this form matches Definition 14 with $x = \tilde{p}$ and $\vartheta_i = \eta_i(z) - 1$ for $i \in [d]$, along with

$$\begin{aligned} \phi_j(\tilde{p}) &= \log \tilde{p}_j \\ A(\eta(z)) &= \log B(\eta(z)) \\ h(\tilde{p}) &= 1. \end{aligned}$$

Therefore the exponential family identities in (9) and (10) give

$$\begin{aligned} \mathbb{E}_{\tilde{p}}[\log \tilde{p}_j] &= \frac{\partial}{\partial \eta_j} \log B(\eta(z)) \\ \text{Cov}(\log \tilde{p}_i, \log \tilde{p}_j) &= \frac{\partial^2}{\partial \eta_i \partial \eta_j} \log B(\eta(z)). \end{aligned}$$

We then find

$$\begin{aligned} \frac{\partial}{\partial \eta_j} \log B(\eta(z)) &= \frac{\partial}{\partial \eta_j} \left[\sum_{k=1}^d \log \Gamma(\eta_k(z)) - \log \Gamma(\tau) \right] \\ &= \psi(\eta_j(z)) - \psi(\tau), \end{aligned}$$

where τ is from (11). Therefore, $\mathbb{E}_{\tilde{p}}[\log \tilde{p}_j] = \psi(\eta_j(z)) - \psi(\tau) = \psi(rs_y(z) + \alpha) - \psi(r + d\alpha)$ and $F(z) = \mathbb{E}_{\tilde{p}}[-\log \tilde{p}_j] = \psi(r + d\alpha) - \psi(rs_y(z) + \alpha)$, which is exactly (4).

Computing the second derivative, we find

$$\begin{aligned}\frac{\partial^2}{\partial \eta_i \partial \eta_j} \log B(\eta(z)) &= \frac{\partial}{\partial \eta_i} [\psi(\eta_j(z)) - \psi(\tau)] \\ &= \delta_{ij} \psi'(\eta_j(z)) - \psi'(\tau).\end{aligned}$$

Evaluating this expression with $i = j = y$ gives $\text{Cov}(\log \tilde{p}_y, \log \tilde{p}_y) = \psi'(rs_y(z) + \alpha) - \psi'(r + d\alpha)$. Since $\text{Var}_{\tilde{p}}[\mathcal{L}(\tilde{p}(z), y)] = \text{Var}_{\tilde{p}}[-\log \tilde{p}_y] = \text{Var}_{\tilde{p}}[\log \tilde{p}_y] = \text{Cov}(\log \tilde{p}_y, \log \tilde{p}_y)$, we have $\text{Var}_{\tilde{p}}[\mathcal{L}(\tilde{p}(z), y)] = \psi'(rs_y(z) + \alpha) - \psi'(r + d\alpha)$, which is exactly (8).

Before proving the result for $\mathbb{E}_{\tilde{p}} \left[\frac{\partial}{\partial z_j} \mathcal{L}(\tilde{p}(z), y) \right]$, we first compute $\frac{\partial}{\partial z_j} \mathbb{E}_{\tilde{p}} [\mathcal{L}(\tilde{p}(z), y)]$. Let $\mathcal{L}_{\text{CE}}(z) = -\log s_y(z)$. Then the partial derivatives with respect to z_j are given by

$$\frac{\partial s_y(z)}{\partial z_j} = s_y(z)(\delta_{yj} - s_j(z)) \quad (12)$$

$$\frac{\partial \mathcal{L}_{\text{CE}}(z)}{\partial z_j} = s_j(z) - \delta_{yj}. \quad (13)$$

Differentiating (7) gives

$$\begin{aligned}\frac{\partial}{\partial z_j} \mathbb{E}_{\tilde{p}} [\mathcal{L}(\tilde{p}(z), y)] &= \frac{\partial}{\partial z_j} [\psi(r + d\alpha) - \psi(rs_y(z) + \alpha)] \\ &= -\psi'(rs_y(z) + \alpha) \cdot r \cdot \frac{\partial s_y(z)}{\partial z_j} \\ &= -r\psi'(rs_y(z) + \alpha) \cdot s_y(z)(\delta_{yj} - s_j(z)) \\ &= rs_y(z)\psi'(rs_y(z) + \alpha) \cdot (s_j(z) - \delta_{yj}) \\ &= \kappa(z; r, \alpha) \frac{\partial \mathcal{L}_{\text{CE}}(z)}{\partial z_j},\end{aligned} \quad (14)$$

where the third line uses (12) and the fifth line uses (13).

Finally, we prove

$$\mathbb{E}_{\tilde{p}} \left[\frac{\partial}{\partial z_j} \mathcal{L}(\tilde{p}(z), y) \right] = \kappa(z; r, \alpha) \frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_j}(z). \quad (15)$$

Since we have already shown in (14) that $\frac{\partial}{\partial z_j} \mathbb{E}_{\tilde{p}} [\mathcal{L}(\tilde{p}(z), y)] = \kappa(z; r, \alpha) \frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_j}$, the equality in (15) follows directly from an application of the Dominated Convergence Theorem, i.e.,

$$\mathbb{E}_{\tilde{p}} \left[\frac{\partial}{\partial z_j} \mathcal{L}(\tilde{p}(z), y) \right] = \frac{\partial}{\partial z_j} \mathbb{E}_{\tilde{p}} [\mathcal{L}(\tilde{p}(z), y)] = \kappa(z; r, \alpha) \frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_j},$$

and the remainder of the proof justifies the application of the Dominated Convergence Theorem (Lieb and Loss, 2001, Theorem 1.8). To emphasize the density's dependence on z we abuse notation and denote the Dirichlet density by $f_z(u) = \frac{1}{B(\eta(z))} \prod_{j=1}^d u_i^{\eta_j(z)-1}$. We have $\mathbb{E}[-\log \tilde{p}_y] = \int_{S^{d-1}} -\log u_y f(u; \eta(z)) du$. To move $\frac{\partial}{\partial z_j}$ inside the integral with the

Dominated Convergence Theorem, we must show that there exists an integrable function $Z : S^{d-1} \rightarrow \mathbb{R}$ such that

$$\left| \frac{\partial}{\partial z_j} [-\log u_y f(u; \eta(z))] \right| \leq Z(u)$$

for all $u \in S^{d-1}$. We first have

$$\begin{aligned} \frac{\partial}{\partial z_j} [f(u; \eta(z))] &= f(u; \eta(z)) \frac{\partial}{\partial z_j} \log f(u; \eta(z)) \\ &= f(u; \eta(z)) \left[-\frac{\partial}{\partial z_j} \log B(\eta(z)) + \sum_{k=1}^d \frac{\partial \eta_k(z)}{\partial z_j} \log u_k \right]. \end{aligned}$$

Using $\log B(\eta(z)) = \sum_{k=1}^d \log \Gamma(\eta_k(z)) - \log \Gamma(\tau)$ then gives $\frac{\partial}{\partial \eta_k} \log B(\eta(z)) = \psi(\eta_k(z)) - \psi(\tau)$, and applying the chain rule gives

$$\frac{\partial}{\partial z_j} \log B(\eta(z)) = \sum_{k=1}^d \frac{\partial \eta_k(z)}{\partial z_j} (\psi(\eta_k(z)) - \psi(\tau))$$

and

$$\frac{\partial}{\partial z_j} [f(u; \eta(z))] = f(u; \eta(z)) \sum_{k=1}^d \frac{\partial \eta_k(z)}{\partial z_j} (\log u_k - \psi(\eta_k(z)) + \psi(\tau)).$$

Therefore

$$\left| \frac{\partial}{\partial z_j} [-\log u_y f(u; \eta(z))] \right| \leq |\log u_y| f(u; \eta(z)) \sum_{k=1}^d \left| \frac{\partial \eta_k(z)}{\partial z_j} \right| (|\log u_k| + |\psi(\eta_k(z))| + |\psi(\tau)|). \quad (16)$$

We now establish that the terms that depend on z are bounded:

1. The density $f(u; \eta(z))$ is bounded:

(a) Since $\eta_k(z) \in [\alpha, \alpha + r]$ and $1/B(\cdot)$ is continuous on $(0, \infty)$, there exists a constant $C_1 < \infty$ such that $\left| \frac{1}{B(\eta(z))} \right| \leq C_1$.

(b) Because $u_k \in (0, 1]$ and $\eta_k(z) \geq \alpha$, we have $u_k^{\eta_k(z)-1} \leq u_k^{\alpha-1}$.

2. We have $\frac{\partial \eta_k(z)}{\partial z_j} = r \frac{\partial s_k(z)}{\partial z_j}$, where $\frac{\partial s_k(z)}{\partial z_j}$ is the derivative of the softmax function, which is bounded, and there exists a constant C_2 such that $\left| \frac{\partial \eta_k(z)}{\partial z_j} \right| \leq C_2$.

3. Since $\eta_k(z) \in [\alpha, \alpha + r]$ and ψ is continuous on $(0, \infty)$, there exists a constant C_3 such that $|\psi(\eta_k(z))| \leq C_3$.

4. The value of $\psi(\tau)$ is constant because $\tau = r + d\alpha$.

Combining items 1-4 in (16) gives the bound

$$\begin{aligned} \left| \frac{\partial}{\partial z_j} [-\log u_y f(u; \eta(z))] \right| &\leq C_1 C_2 |\log u_y| \left(d(C_3 + |\psi(\tau)|) + \sum_{j=1}^d |\log u_j| \right) \prod_{i=1}^d u_i^{\alpha-1} \\ &\leq C |\log u_y| \left(1 + \sum_{j=1}^d |\log u_j| \right) \prod_{i=1}^d u_i^{\alpha-1}, \end{aligned}$$

where $C = C_1 C_2 (d(C_3 + |\psi(\tau)|) + 1)$.

Now we show that this dominating function has a finite integral over the simplex. We use Young's inequality, namely

$$ab \leq \frac{a^2 + b^2}{2},$$

with $a = |\log u_y|$ and $b = 1 + \sum_{j=1}^d |\log u_j|$ to find

$$\begin{aligned} C |\log u_y| \left(1 + \sum_{j=1}^d |\log u_j| \right) \prod_{i=1}^d u_i^{\alpha-1} &\leq \frac{C}{2} \left(|\log u_y|^2 + \left(1 + \sum_{j=1}^d |\log u_j| \right)^2 \right) \prod_{i=1}^d u_i^{\alpha-1} \\ &\leq \frac{C}{2} \left(|\log u_y|^2 + (d+1) \left(1 + \sum_{j=1}^d |\log u_j|^2 \right) \right) \prod_{i=1}^d u_i^{\alpha-1}, \quad (17) \end{aligned}$$

where the second inequality applies Cauchy-Schwarz. For the function on the right-hand side of (17) to be integrable, it suffices to show that

$$\int_{S^{d-1}} |\log u_j|^2 \prod_{i=1}^d u_i^{\alpha-1} du < \infty \quad (18)$$

for all $j \in [d]$. In the scalar case, i.e., for $u \in (0, 1]$ rather than $u \in S^{d-1}$, the corresponding integrals are finite because

$$\int_0^1 u^{\alpha-1} du = \frac{1}{\alpha} < \infty$$

and

$$\int_0^1 |\log u|^2 u^{\alpha-1} du = \int_0^1 (-\log u)^2 u^{\alpha-1} du = \frac{2}{\alpha^3} < \infty.$$

For each $j \in [d]$, the simplex integral in (18) has the same form as one of the scalar integrals above. Thus, the dominating function in (17) is integrable and the Dominated Convergence Theorem applies.

C.1 Asymptotic Analysis and Empirical Results for Section 3.5

We perform a brief asymptotic analysis to characterize how γ_{eff} in (6) varies with α . First, we account for the fact that r^* is constrained by the condition $h(r) \leq \epsilon$ from (1). We

α	r^*	r^*/α
0.1	0.0087	0.0870
0.3	0.0260	0.0867
0.5	0.0433	0.0866
1	0.0864	0.0864
2	0.1580	0.0790
3	0.2349	0.0783

Table 6: Values of r^* found with (1) for CIFAR10/ResNet-18, with $\epsilon = 1$, $\delta = 10^{-5}$, a batch size of 250, and $T = 20000$ steps.

approximate r^* as a linear function of α of the form $c(\epsilon)\alpha$. Table 6 shows that at $\epsilon = 1$, the behavior of r^* is roughly linear in α with $c(1) \approx 0.08$.

Substituting the linear approximation into (5) and (3) and using $\psi'(x) \approx x^{-2}$ as $x \rightarrow 0$ and $\psi'(x) \approx x^{-1}$ as $x \rightarrow \infty$ yields the two regimes

$$\kappa(z; r, \alpha) \approx \begin{cases} \alpha^{-1}, & \alpha \ll 1 \\ \text{const}, & \alpha \gg 1, \end{cases} \quad \text{Var}_{\tilde{p}}[\mathcal{L}(z; \tilde{p})] \approx \begin{cases} \alpha^{-2}, & \alpha \ll 1 \\ \alpha^{-1}, & \alpha \gg 1. \end{cases}$$

Both the scaling factor $\kappa(z; r, \alpha)$ and the per-step variance $\text{Var}_{\tilde{p}}[\mathcal{L}(z; \tilde{p})]$ shrink as α grows, but the variance shrinks faster. Thus, as α increases, Algorithm 1 takes smaller steps on average but also experiences substantially less privacy-induced variance. This behavior makes runs with larger α less noisy and allows for stability at larger learning rates.

C.2 Learning Rate Values for Figure 2

Here we provide the learning rate values tested for each α value in Figure 2. For $\alpha \in \{0.1, 0.2, 0.3, 0.5\}$ we use 11 values, namely $\gamma \in \{0.01, 0.025, 0.05, 0.075, 0.1, 0.2, 0.3, 0.4, 0.5, 0.75, 1\}$. For $\alpha \in \{1, 2, 3\}$ we use 13 values, namely $\gamma \in \{0.05, 0.075, 0.1, 0.15, 0.2, 0.3, 0.5, 1, 1.5, 2, 3, 5, 10\}$. For $\alpha \in \{5, 10\}$ we use 11 values, namely $\gamma \in \{0.1, 0.2, 0.5, 0.75, 1, 2, 3, 4, 5, 6, 7\}$.

Appendix D. Implementation Details

D.1 Data Sets

We evaluate our framework on the following image classification data sets:

- CIFAR10 (Krizhevsky et al., 2009) provides 32×32 color images categorized into 10 distinct classes. The data set is partitioned into 50,000 images for training and 10,000 for testing.
- MNIST (LeCun et al., 1998) consists of 28×28 grayscale images of handwritten digits divided into 10 classes. The standard benchmark includes 60,000 training samples and 10,000 testing samples.

Data set	α	$\epsilon = 0.5$	$\epsilon = 1$	$\epsilon = 2$	$\epsilon = 3$	$\epsilon = 4$	$\epsilon = 6$	$\epsilon = 8$
CIFAR10	3.0	—	0.235	0.404	0.559	0.679	0.825	0.903
MNIST	3.0	0.136	0.277	0.472	—	0.711	—	—
DermaMNIST	3.0	—	0.137	—	—	0.645	—	0.856 _($\epsilon=7.42$)
SVHN	3.0	—	0.284	0.475	0.575	0.717	0.930	0.964
FashionMNIST	3.0	0.136	0.277	0.472	—	0.711	—	—

 Table 7: Dirichlet parameters r^* and α for each data set and each value of ϵ when $\delta = 10^{-5}$.

- DermaMNIST (Yang et al., 2021, 2023), part of the broader MedMNIST collection, focuses on classifying dermoscopic images into 7 different types of skin lesions. The images are standardized to 28×28 pixels, with the data set split into 7,007 training, 993 validation, and 2,005 test examples.
- FashionMNIST (Xiao et al., 2017) consists of Zalando’s clothing articles. It shares the same 10-class structure, 28×28 grayscale format, and 60,000/10,000 train-test split as MNIST.
- SVHN (Netzer et al., 2011) (Street View House Numbers) contains 32×32 RGB images of cropped digits captured from real-world street signs. It covers 10 classes and provides 73,257 training images alongside 26,032 testing images in its core data set.

D.2 Dirichlet Parameters

In Table 7, we report the Dirichlet parameters r^* and α for each data set and each privacy budget when using $\delta = 10^{-5}$. Each value of r^* is computed using (1).

Appendix E. Additional Results on DermaMNIST

We follow Hölzl et al. (2022) and report validation accuracy in Table 8. Our work outperforms the DP-SGD baseline (Tang et al., 2023) and Hölzl et al. (2022) across all evaluated values of ϵ .

Method	Model	$\epsilon = 1$	$\epsilon = 4$	$\epsilon = 7.42$	$\epsilon = \infty$
DP-SGD (Tang et al., 2023)	ResNet-9	69.00	71.78	74.08	77.27
Hölzl et al. (2022)	ResNet-9	—	—	74.17	77.84
This work	ResNet-9	74.98 ± 2.04	76.27 ± 1.22	76.49 ± 1.45	79.26

 Table 8: Validation accuracy (%) on DermaMNIST under $\delta = 10^{-5}$ across 5 independent runs for each ϵ .

Method	Model	$q = 0.001$	$q = 0.01$	$q = 0.02$	$q = 0.04$
This work	ResNet-18	89.41 ± 3.63	82.94 ± 2.73	71.18 ± 5.53	56.49 ± 3.51

Table 9: Test accuracy (%) of this work using different sampling rates on CIFAR10 across 5 independent runs for $(4.0, 10^{-5})$ -differential privacy.

Method	Model	$\epsilon = 1$	$\epsilon = 2$	$\epsilon = 3$	$\epsilon = 4$	$\epsilon = 6$	$\epsilon = 8$	$\epsilon = \infty$
This work	ResNet-18	86.78 ± 0.66	87.50 ± 0.61	87.99 ± 0.25	88.27 ± 0.39	88.57 ± 0.57	88.62 ± 0.24	94.58

Table 10: Test accuracy (%) of Algorithm 1 using the Adam optimizer on CIFAR10 across 5 independent runs for each ϵ .

Appendix F. Experiments with Different Sampling Rates

We use the SGD optimizer with learning rate $\gamma = 0.1$, $T = 20000$ steps, no momentum, and with random crop and horizontal flip on CIFAR10 with ResNet-18 as a baseline. We then investigate the effects of different sampling rates under $(4, 10^{-5})$ -differential privacy with Dirichlet parameter $\alpha = 3.0$. As shown in Table 9, we observe that smaller sampling rates improves the accuracy of private training with Algorithm 1, while large sampling rates such as $q = 0.04$ lead to worse accuracy. This degradation in accuracy is caused by privacy amplification effects, since when the sampling rate is large, the privacy-utility tradeoff benefits less from privacy amplification. In practice, a small sampling rate is desired when training with Algorithm 1.

Appendix G. Results on Different Optimizer

We also test our private training framework using ResNet-18 on CIFAR10 with the Adam optimizer (Kingma and Ba, 2014) using learning rate $\gamma = 0.001$, sampling rate $q = 0.005$, $T = 20000$ steps, no momentum, $\delta = 10^{-5}$, and $\alpha = 3.0$.

Table 10 shows the privacy-utility tradeoff for Algorithm 1 using the Adam optimizer. The accuracy increases as ϵ increases, which follows the same trend as using the SGD optimizer. Notably, even at strongest privacy level of $\epsilon = 1$, the accuracy is 86.78%. These results illustrate that Algorithm 1 is optimizer-agnostic in the sense that any optimizer can be used without harming privacy, while also maintaining high accuracy.

References

- M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 308–318, 2016.

- M. Bazarbash. *Fintech in financial inclusion: machine learning applications in assessing credit risk*. International Monetary Fund, 2019.
- S. Bhagat, U. Weinsberg, S. Ioannidis, and N. Taft. Recommending with an agenda: Active learning of private attributes using matrix factorization. In *Proceedings of the 8th ACM conference on recommender systems*, pages 65–72, 2014.
- R. P. Brent. An algorithm with guaranteed convergence for finding a zero of a function. *The computer journal*, 14(4):422–425, 1971.
- Z. Bu, Y.-X. Wang, S. Zha, and G. Karypis. Automatic clipping: Differentially private deep learning made easier and stronger. *Advances in Neural Information Processing Systems*, 36:41727–41764, 2023.
- R. I. Busa-Fekete, A. M. Medina, U. Syed, and S. Vassilvitskii. Label differential privacy and private training data release. In *International Conference on Machine Learning*, pages 3233–3251. PMLR, 2023.
- C. L. Canonne, G. Kamath, and T. Steinke. The discrete gaussian for differential privacy. *Advances in Neural Information Processing Systems*, 33:15676–15688, 2020.
- A. Cheng, J. Wang, X. S. Zhang, Q. Chen, P. Wang, and J. Cheng. Dpnas: Neural architecture search for deep learning with differential privacy. In *Proceedings of the AAAI conference on artificial intelligence*, volume 36, pages 6358–6366, 2022.
- W. L. Croft, J.-R. Sack, and W. Shi. Obfuscation of images via differential privacy: From facial images to general images. *Peer-to-Peer Networking and Applications*, 14(3):1705–1733, 2021.
- S. De, L. Berrada, J. Hayes, S. L. Smith, and B. Balle. Unlocking high-accuracy differentially private image classification through scale. *arXiv preprint arXiv:2204.13650*, 2022.
- C. Dwork, A. Roth, et al. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014.
- H. Esfandiari, V. Mirrokni, U. Syed, and S. Vassilvitskii. Label differential privacy via clustering. In *International Conference on Artificial Intelligence and Statistics*, pages 7055–7075. PMLR, 2022.
- L. Fan. Image pixelization with differential privacy. In *IFIP Annual Conference on Data and Applications Security and Privacy*, pages 148–162. Springer, 2018.
- M. Fredrikson, S. Jha, and T. Ristenpart. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, pages 1322–1333, 2015.
- K. Ganju, Q. Wang, W. Yang, C. A. Gunter, and N. Borisov. Property inference attacks on fully connected neural networks using permutation invariant representations. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, pages 619–633, 2018.

- B. Ghazi, N. Golowich, R. Kumar, P. Manurangsi, and C. Zhang. Deep learning with label differential privacy. *Advances in neural information processing systems*, 34:27131–27145, 2021.
- B. Ghazi, P. Kamath, R. Kumar, E. Leeman, P. Manurangsi, A. Varadarajan, and C. Zhang. Regression with label differential privacy. In *The Eleventh International Conference on Learning Representations*, 2023.
- K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- F. A. Hölzl, D. Rueckert, and G. Kaissis. Bridging the gap: Differentially private equivariant deep learning for medical image analysis. *arXiv preprint arXiv:2209.04338*, 2022.
- J. Hsu, M. Gaboardi, A. Haeberlen, S. Khanna, A. Narayan, B. C. Pierce, and A. Roth. Differential privacy: An economic method for choosing epsilon. In *2014 IEEE 27th Computer Security Foundations Symposium*, pages 398–410. IEEE, 2014.
- D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- A. Krizhevsky, G. Hinton, et al. Learning multiple layers of features from tiny images. *Technical Report*, 2009.
- Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- E. H. Lieb and M. Loss. *Analysis*, volume 14. American Mathematical Soc., 2001.
- Y. Luo, T. Hu, X. Ouyang, J. Liu, Q. Fu, S. Qin, Z. Min, and X. Lin. Dpo-face: Differential privacy obfuscation for facial sensitive regions. *Computers & Security*, 154:104434, 2025.
- M. Malek Esmaeili, I. Mironov, K. Prasad, I. Shilov, and F. Tramer. Antipodes of label differential privacy: Pate and alibi. *Advances in neural information processing systems*, 34:6934–6945, 2021.
- L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov. Exploiting unintended feature leakage in collaborative learning. In *2019 IEEE symposium on security and privacy (SP)*, pages 691–706. IEEE, 2019.
- I. Mironov. Rényi differential privacy. In *2017 IEEE 30th computer security foundations symposium (CSF)*, pages 263–275. IEEE, 2017.
- I. Mironov, K. Talwar, and L. Zhang. Rényi differential privacy of the sampled gaussian mechanism. *arXiv preprint arXiv:1908.10530*, 2019.
- I. A. Monir and G. Ghinita. Differentially-private neural network training with private features and public labels. In *International Conference on Big Data Analytics and Knowledge Discovery*, pages 208–222. Springer, 2024.

- Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng. Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*, 2011.
- K. Packhäuser, S. Gündel, N. Münster, C. Syben, V. Christlein, and A. Maier. Deep learning-based patient re-identification is able to exploit the biometric nature of medical chest x-ray data. *Scientific Reports*, 12(1):14851, 2022.
- N. Papernot, A. Thakurta, S. Song, S. Chien, and Ú. Erlingsson. Tempered sigmoid activations for deep learning with differential privacy. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 9312–9321, 2021.
- A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- D. Ponnoprat. Dirichlet mechanism for differentially private kl divergence minimization. *Transactions on Machine Learning Research*, 2023.
- M. Rigaki and S. Garcia. A survey of privacy attacks in machine learning. *ACM Computing Surveys*, 56(4):1–34, 2023.
- T. Sander, P. Stock, and A. Sablayrolles. Tan without a burn: Scaling laws of dp-sgd. In *International Conference on Machine Learning*, pages 29937–29949. PMLR, 2023.
- R. Shokri, M. Stronati, C. Song, and V. Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*, pages 3–18. IEEE, 2017.
- C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.
- X. Tang, A. Panda, V. Schwag, and P. Mittal. Differentially private image classification by learning priors from random processes. *Advances in neural information processing systems*, 36:35855–35877, 2023.
- F. Tramer and D. Boneh. Differentially private learning needs better features (or much more data). In *International Conference on Learning Representations*, 2020.
- M. J. Wainwright and M. I. Jordan. Graphical models, exponential families, and variational inference. *Foundations and Trends® in Machine Learning*, 1(1-2):1–305, 2008.
- X. Wang, Y. Peng, L. Lu, Z. Lu, M. Bagheri, and R. M. Summers. Chestx-ray8: Hospital-scale chest x-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2097–2106, 2017.
- Y.-X. Wang, B. Balle, and S. P. Kasiviswanathan. Subsampled rényi differential privacy and analytical moments accountant. In *The 22nd international conference on artificial intelligence and statistics*, pages 1226–1235. PMLR, 2019.

- H. Xiao, K. Rasul, and R. Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- H. Xue, B. Liu, M. Ding, T. Zhu, D. Ye, L. Song, and W. Zhou. Dp-image: Differential privacy for image data in feature space. *arXiv preprint arXiv:2103.07073*, 2021.
- J. Yang, R. Shi, and B. Ni. Medmnist classification decathlon: A lightweight automl benchmark for medical image analysis. In *IEEE 18th International Symposium on Biomedical Imaging (ISBI)*, pages 191–195, 2021.
- J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni. Medmnist v2-a large-scale lightweight benchmark for 2d and 3d biomedical image classification. *Scientific Data*, 10(1):41, 2023.
- A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao, G. Cormode, and I. Mironov. Opacus: User-friendly differential privacy library in PyTorch. *arXiv preprint arXiv:2109.12298*, 2021.
- S. Zagoruyko and N. Komodakis. Wide residual networks. *arXiv preprint arXiv:1605.07146*, 2016.
- Y. Zhu and Y.-X. Wang. Poission subsampled rényi differential privacy. In *International Conference on Machine Learning*, pages 7634–7642. PMLR, 2019.